

从长列表看高德地图AJX容器能力演进

一、前言

你是否还记得2015年前后的移动互联网？

打开手机APP可以几块钱 打车、吃饭、看电影...

各种O2O、共享经济，写字楼下有扫码一条街（扫码下载地推人员推广的APP就送小礼物）

那是个移动互联网井喷式发展的年代，手机APP忙着抢夺每一位用户

彼时高德地图客户端已是成熟的APP，移动客户端的技术栈以Android、iOS原生为主，产品迭代依赖APP发版成了高德业务发展的瓶颈。

那一年，*React Native* 刚刚开源，那一年 *weex* 还是1.0，那一年还没有 *flutter*

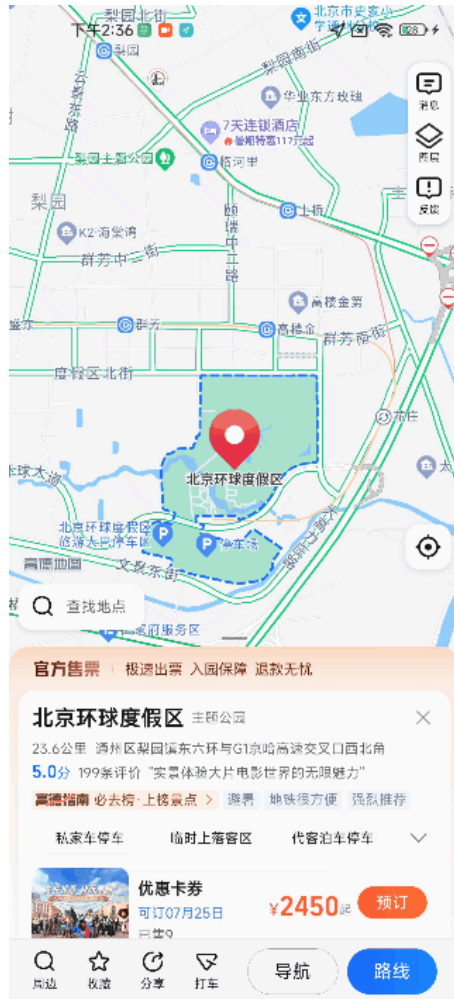
那一年，高德地图对动态方案慎重选型后决定自研，AJX容器应运而生。 [《揭秘高德地图动态化跨端容器》](#)

二、起步

AJX容器是以 **JS引擎 搭配 Android、IOS原生组件渲染的跨端动态化容器**，目标是服务好高德快速发展的业务，在实现动态发版快速迭代的前提下，提供对标原生开发的性能&交互体验。详细架构 [《一文读懂高德地图AJX容器架构原理》](#)

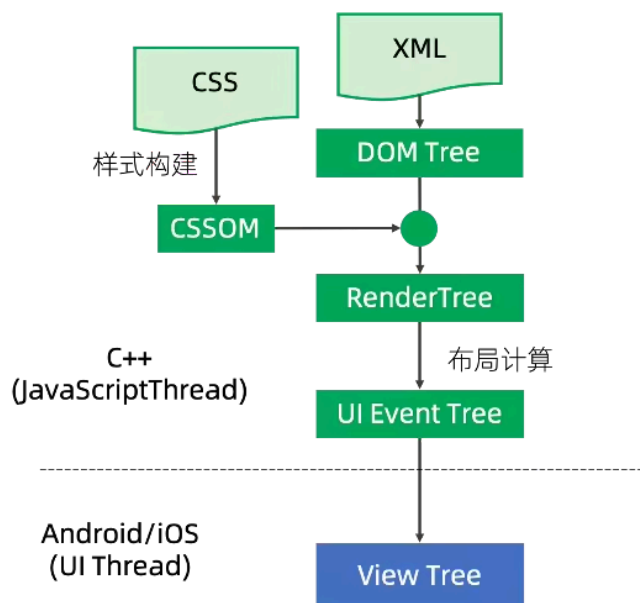
落地POI页面

为了验证AJX容器的可行性，选择了<POI页面>作为第一个业务场景落地。POI几乎是每位用户都会使用的场景，对体验要求高，功能涵盖列表的主流交互需求。



POI页面展示

最初AJX是否可以高质量实现列表交互，大家心里并没有谱，相较于客户端原生开发，AJX容器的布局计算在JS线程，渲染在UI线程



布局计算-示意图

主要难点如何处理跨线程的手势交互：

- 支持上下**两段式滑动**，以便露出地图
- 拖动列表的时候，头图支持**差速动画**

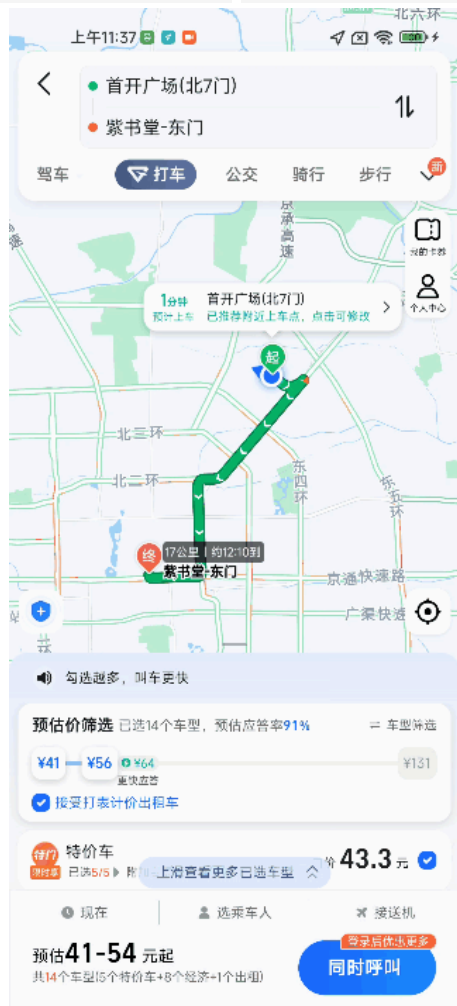
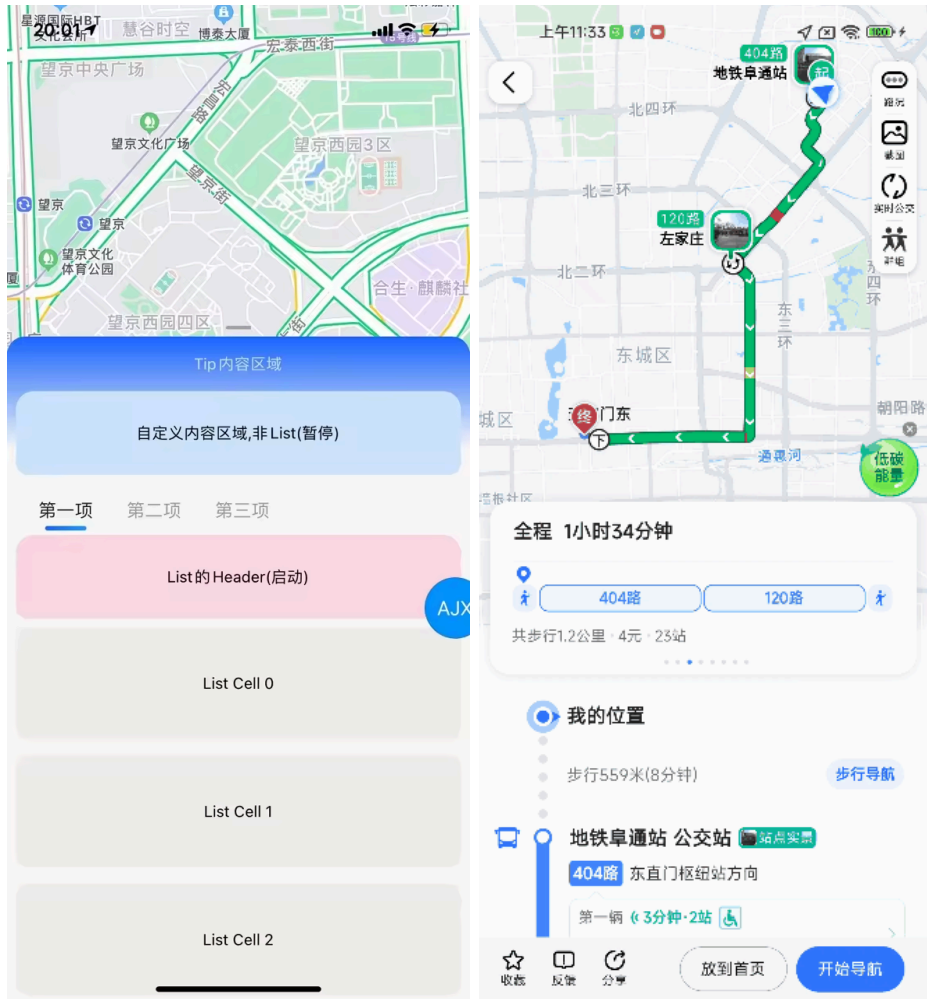
如果把交互功能封装在原生组件，看上去是一条捷径，但也意味着组件与业务场景绑定，丧失了通用性和动态化能力，最终AJX容器提供了两个**通用能力**，在JS层实现了POI的交互需求

- 关联动画**，不依赖JS线程的动画能力
- 模拟滚动**，赋予列表滚动交互更多的可能性

有兴趣详细了解，强烈推荐阅读@毕哥 [《高德地图AJX动效前世今生及思索下一代交互形态》](#)

举一反三

类似于POI的“半图半表(地图+列表)”的场景高德有许多，通过POI落地的经验，演进出**通用滑动容器** <[PanContainer](#)>，它是在前端对AJX容器基础组件、关联动画等通用能力的封装，已被普遍用于高德地图各业务场景。



三、发展

直至今日AJX容器已承载了客户端近乎100%的核心业务，JS代码量600+万行，过程中业务对AJX容器提出了更多的挑战。

1. 首帧渲染优化

上面提到，AJX容器布局计算是异步的，因此打开页面时需要先加载JS Context以及JS Framework，然后JS线程开始计算布局发送UI事件到主线程绘制，这就导致AJX在UI首帧渲染速度对比原生开发处于劣势。

最初的阶段，为了优化POI页面的首帧渲染速度，除了在业务层面对页面内容做分批加载外，还从两方面着手：

- APP运行期始终在内存预留一个空闲的JS Context给POI场景使用，确保打开页面第一时间开始布局计算。
- 依赖原生开发在POI页面添加占位组件，当JS计算的布局返回后再由AJX组件替代展示。

最终以上优化项演进成AJX容器的**通用能力**，解除了对原生开发的依赖：

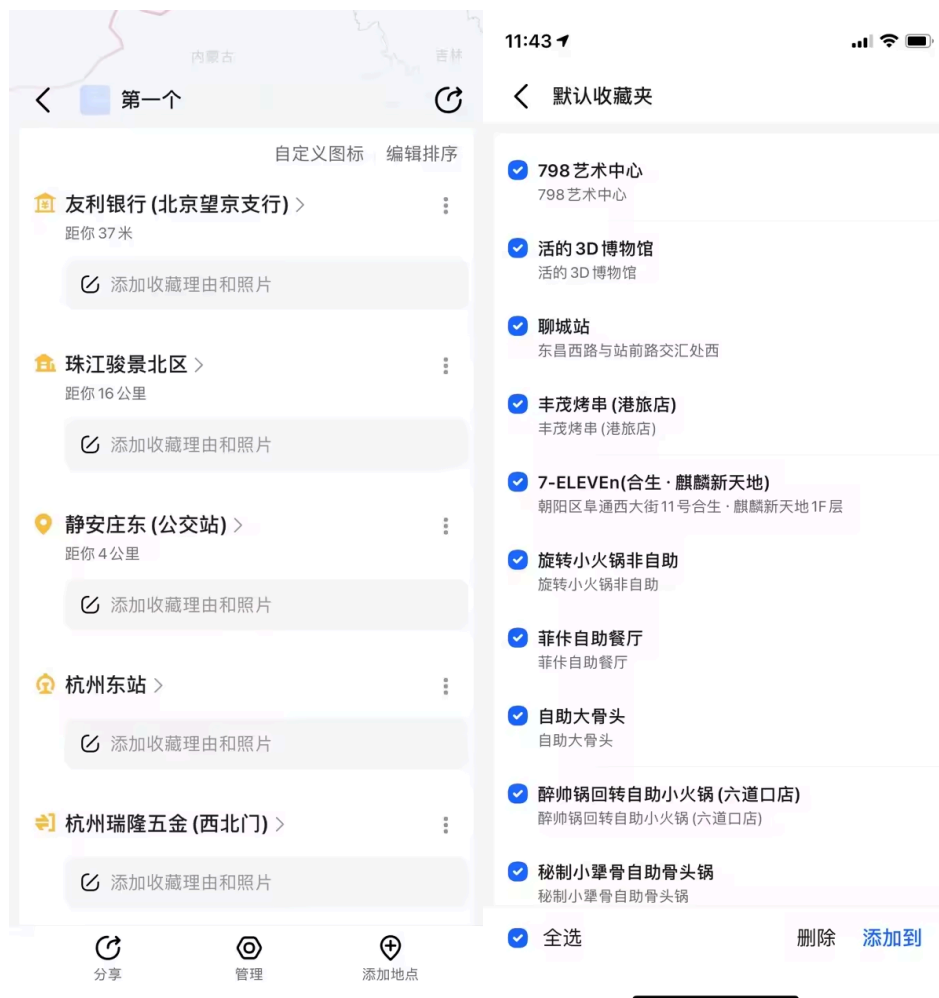
-

[引擎预加载能力](#)

- Splash骨架屏能力**

2. 超长的列表

通常我们建议超长列表做分页加载。但诸如高德的收藏夹业务，需要一次性加载并支持批量修改，这就无法在产品交互上通过分页加载解题。问题：当用户收藏的点越多，list初始化的耗时就越久，同时批量修改也会卡顿。



收藏夹-支持批量修改

思路：虽然无法从产品交互做分页展示，但通过**按需加载**解题的方向一定没错

解题：采用了在前端实现按需加载，按需进行批量更新的方式，结合native及C++引擎提供的模板识别及缓存section的能力，这样利用最小的成本给长列表带来了很大的性能提升，By 击浪。《[高德地图长列表优化实践](#)》

优化效果：

3. 平滑滚动

滚动帧率是列表性能体验的重要一环。<list>组件充分利用原生组件RecyclerView、UITableView的视图复用能力，并且受益于异步布局计算，<list>在滚动前就已经知道每个View的宽高位置，滚动时UI

线程可跳过layout步骤，只需负责渲染即可。所以理论上<list>滚动帧率>=原生开发的列表滚动帧率。

理想是美好的，现实是骨感的，由于前端和客户端的开发习惯不同，会引入一些额外的性能成本

- 前端习惯大量嵌套层级，每嵌套一层div，原生渲染就多一层负担。经统计使用列表的页面层级大都在15-20层，是早期困扰安卓设备性能体验的一大问题。在POI及早期接入的业务中，只能从JS层降低div嵌套。后续也有调研在AJX容器内把View层级拉平的方案，但经过多次review我们判断拉平层级并不具备良好的通用性，会极大的约束手势、动画等能力，因此最终并未上线。随着安卓设备性能的突飞猛进，直到今日，20层以内的渲染层级已不是性能卡点。

- 前端习惯使用display:none，例如常见的列表展开/收起功能。display:none的节点虽然并不会渲染，但其所有style仍然会被解析，在列表滚动时该节点所处位置进入屏幕区域内样式就会生效。例如早期在酒店列表，每一个酒店cell下隐藏了大量(10个+)折叠状态的房间cell，列表滚动起来卡卡的，就是因为对大量不可见view设置style导致。优化后<list>组件在滚动时会忽略掉display:none的cell，卡顿问题得以解决。

4. 瀑布流

最初<list>组件仅支持单列列表，这显然无法满足高德地图的业务需求，例如附近页：



附近页-瀑布流列表

AJX容器基于<list>实现了<waterfall>组件，业务只需要在section上添加"column-count=n"字段即可轻实现n列布局的瀑布流，添加/删除cell也可以轻松对标原生体验的位移动画效果，并且<waterfall>继承<list>的全部能力。

5. 节点曝光

<list>组件提供了appeared事件回调，当cell第一次进入屏幕时触发回调，经常被用作曝光埋点。但随着高德业务的发展，对于组件曝光的精细度要求变高。原有的appeared事件回调的最小单位是cell节点，并且露出1px就会触发回调，已经无法满足业务的需求。

AJX容器实现了标准 API [IntersectionObserver](#) 满足精细化曝光埋点的需求。

四、结语

已经度过五年陈的AJX容器，承载了高速发展的高德业务。得益于选择自研容器的道路，使得AJX容器与高德业务足够紧密，能根据不同场景深度定制优化，也能对地图产品特性封装适合高德业务的通

用能力。AJX容器经受住了复杂业务交互、性能体验等需求，足以说明以原生组件渲染的动态化容器的价值。AJX容器未来会持续打磨，着力于服务好高德业务，围绕高德地图的产品定位继续做深做透。